

JAVADOC

Genera páginas HTML de documentación del API a partir de ficheros con código fuente Java.

Sintaxis

```
javadoc [opciones] nombre_de_paquete | clase1.java clase2.java
```

Descripción

javadoc genera documentación API en formato HTML para el paquete especificado o para los archivos fuentes individuales en Java en la línea de comandos. Javadoc genera un fichero *.html* por cada fichero *.java* y paquete que encuentra. También genera la jerarquía de clases (*tree.html*) y un índice con todos los miembros que ha detectado (*AllNames.html*). Como argumento a **javadoc**, se le puede indicar una serie de paquetes o ficheros Java con los que se desea generar la documentación.

Doclets

Se puede configurar el contenido y el formato de salida que se va a generar con **javadoc** a través de *doclets*. Un **doclet** es un programa Java escrito utilizando el Java Doclet API, que especifica el contenido y formato de la salida que ha de generar **javadoc**. Se pueden escribir doclets para generar cualquier tipo de salida de texto, ya sea HTML, SGML, RTF o MIF. JavaSoft solamente proporciona el doclet estándar que genera una salida HTML semejante a la de los JDK anteriores. Sin embargo, a través de los doclets se pueden realizar tareas que nada tengan que ver con la documentación; por ejemplo, un doclet para diagnóstico podría comprobar que todas las clases miembro de las clases sobre las que está operando **javadoc** tienen los comentarios de documentación.

Cuando no se indica ningún doclet específico en la línea de comandos, **javadoc** utilizará el doclet estándar que genera una salida HTML semejante a la de las versiones anteriores del JDK, aunque incluso este doclet incorpora algunas opciones más que sus antecesores en el JDK.

Comentar el código fuente

Como **javadoc** comprueba automáticamente clases, interfaces, métodos y declaraciones de variables, se puede añadir documentación adicional a través de comentarios de documentación en el código fuente. Estos comentarios pueden incluir marcas HTML. Un comentario de documentación está formado por todos los caracteres incluidos entre */*** que indica el comienzo del comentario y **/* que indica el final. En todas las líneas intermedias los caracteres *** a la izquierda se ignoran, y también todos los espacios y tabuladores que precedan a ese carácter ***.

```
/**  
 * Este es un comentario de documentación  
 */
```

HTML estándar

Se pueden incluir marcas HTML dentro de un comentario de documentación, aunque no deben utilizarse marcas de cabecera como <H1> y <H2>, o líneas horizontales <HR>, porque **javadoc** crea una estructura completa para el documento y estas marcas interfieren con el formato general de ese documento.

Marcas de javadoc

javadoc reconoce marcas especiales cuando recorre los comentarios de documentación. Estas marcas permiten autogenerar una documentación completa y bien formateada del API a partir del código fuente. Las marcas comienzan siempre con el signo @. Estas marcas deben situarse al principio de la línea (tener en cuenta que todo lo que haya hasta el primer carácter * se ignora) y todas las marcas con el mismo nombre deben agruparse juntas dentro del comentario de documentación.

Marcas de documentación de clases e interface

@see nombre_de_clase

Añade un link a la clase en la zona "See Also". Por ejemplo:

```
@see java.lang.String
@see String
@see String#equals
@see java.lang.Object#wait(int)
@see Character#MAX_RADIX
@see <A HREF="spec.html">Especif. Java</A>
```

El carácter # separa el nombre de una clase del nombre de uno de sus campos, métodos o constructores. Un comentario de documentación puede incluir más de una marca @see.

@version texto-version

Añade una entrada "Version". El texto no tiene que tener formato especial. Un comentario de documentación puede incluir más de una marca @version.

@author texto-autor

Añade una entrada "Author". El texto no tiene que tener formato especial. Un comentario de documentación puede incluir más de una marca @author.

@since texto

Este texto no tiene una estructura especial. Se utiliza para indicar desde qué fecha o desde qué versión se ha introducido el cambio o característica que indica el texto.

@deprecated texto

Añade un comentario indicando que no debería utilizarse la función o método, porque puede dejar de ser soportada por el API. La convención que se sigue es indicar en el texto la función o método por quien se ha sustituido. Por ejemplo:

`@deprecated Replaced by setBounds(int,int,int,int)`

Si el miembro está ya obsoleto y eliminado, el texto que sigue al tag `@deprecated` debe ser "No replacement".

Ejemplo de comentario de una clase:

```
/**
 * Clase que presenta una ventana en la pantalla.
 * Por ejemplo:
 * <PRE>
 *     Window ventana = new Window( padre );
 *     Ventana.show();
 * </PRE>
 *
 * @see      awt.BaseWindow
 * @see      awt.Button
 * @version  1.3 15 Ene 97
 * @author   Agustin Froufe
 */
class Window extends BaseWindow {
    . . .
}
```

Marcas de documentación de campos

La única marca especial que se puede incluir es la marca `@see`.

Ejemplo de comentario de una clase:

```
/**
 * Coordenada X de la ventana
 * @see window#1
 */
int x=1234567;
```

Marcas de documentación de constructores y métodos

Pueden ser marcas `@see` y además:

`@param parametro descripcion`

Añade un parámetro a la sección "Parameters". La descripción puede continuar en la línea siguiente.

`@return descripcion`

Añade una sección "Return", que debe contener la descripción del valor de retorno.

`@exception nombre_de_la_clase descripcion`

Añade una entrada "Throws", que contiene el nombre de la excepción que puede ser lanzada por el método. La excepción estará enlazada con su clase en la documentación.

`@see nombre_de_clase`

Añade un link a la clase en la zona "See Also".

`@since texto`

Indica desde qué fecha o desde qué versión se ha introducido el cambio o característica que indica el texto.

`@deprecated texto`

Indica que no debería utilizarse la función o método, porque puede dejar de ser soportada por el API.

Ejemplo de comentario de un método:

```
/**
 * Devuelve el carácter de la posición indicada
entre
 * <TT>0</TT> y <TT>length()-1</TT>
 * @param indice La posición del carácter a obtener
 * @return       El carácter situado en la posición
 * @exception    StringIndexOutOfBoundsException
 *               Se prodcue cuando el indice no
está en
 *               el rango <TT>0</TT> a
<TT>length()-1</TT>
 */
public char charAt( int indice ) {
    . . .
}
```

Opciones

Se puede utilizar un argumento que comience por el carácter arroba, @, para indicar que las opciones se encuentran en un fichero, cada argumento en una línea. Estos argumento serán insertados automáticamente en la línea de comandos en la posición en que se haya indicado en el argumento `@fichero`.

javadoc ahora utiliza doclets para determinar su salida. Si no se indica un doclet específico a través de la opción `-doclet`, **javadoc** utilizará el doclet estándar. **javadoc** proporciona un conjunto de opciones que se pueden utilizar con cualquier doclet, *opciones de javadoc*; y también proporciona opciones adicionales a utilizar con el doclet estándar, *opciones con doclet estándar*.

Opciones de javadoc

-doclet *fichero*

Especifica el fichero que contiene la configuración del formato de salida que se desea generar. Si no se especifica, se generará una salida con formato *html* estándar.

-sourcepath *path*

Especifica el camino de búsqueda de los ficheros fuente *.java*. No afecta a la carga de los ficheros de clases *.class*.

-classpath *path*

No es conveniente utilizar esta opción, porque no es necesaria habitualmente; es mejor utilizar **-sourcepath** para indicar donde se encuentran los ficheros .java a documentar. Especifica el camino que **javadoc** usa para encontrar los ficheros .java. También indica los directorios desde los que **javadoc** carga sus propios ficheros .class. Sobreescrive el establecido por defecto o la variable de entorno CLASSPATH si ésta ha sido establecida anteriormente. Los directorios en la variable CLASSPATH son separados con punto y coma (;). El formato general para el *path* es:

```
.;<tu_camino>
```

Por ejemplo:

```
.;C:\users\afq\javasrc
```

-encoding *nombre*

Especifica el tipo de codificación del fichero fuente, como EUCJIS\SJIS. Si no se especifica, se usa en conversor de defecto de la plataforma.

-Jflag

Pasa *flag* directamente al sistema operativo. Por ejemplo, si es necesario que limitemos la memoria utilizada por el sistema a 32 megabytes para generar la documentación, podríamos utilizar este flag de la forma siguiente:

```
javadoc -J-Xmx32m -J-Xms32m &lt;clases> ...
```

-package

Imprime paquetes y clases y miembros públicos y protected.

-private

Muestra todo tipo de clases y sus miembros.

-protected

Muestra solamente las clases públicas y protected y sus miembros. Es la opción de defecto.

-public

Muestra solamente clases y miembros públicos.

-verbose

Sin esta opción aparecen mensajes de carga de ficheros, generación de documentación (un mensaje por fichero) y ordenación. Con esta opción se muestran mensajes adicionales especificando el número de milisegundos empleado en generar la documentación de cada fichero Java.

Opciones con doclet estándar

-author

Incluye las marcas @author, que por defecto son omitidos.

-d *directorio*

Especifica un directorio en donde **javadoc** almacena los archivos HTML que genera. (La "d" significa "destino"). El directorio puede ser absoluto o relativo al directorio de trabajo. Por defecto se

utiliza el directorio actual.

-linkall

Crea links a la documentación del API para todas las clases que referencien algún documento del API que se está generando. Por ejemplo, si **javadoc** está corriendo sin esta opción sobre el fichero `Graphics.java` del paquete `java.awt`, la línea de comandos sería:

```
javadoc C:\java\awt\Graphics.java
```

La *signature* para el método `Graphics.toString()` en el html generado por **javadoc** tendría una apariencia como:

```
<pre>
  public String toString()
</pre>
```

No se generan enlaces a la documentación del API del JDK para la clase **String** no está incluida como entrada en el comando de llamada a **javadoc**. Sin embargo, si ahora se emplea la opción **-linkall**:

```
javadoc -linkall C:\java\awt\Graphics.java
```

sí se genera un enlace a la documentación del API para la clase `String`:

```
<pre>
  public<a href="java.lang.String.html#_top_">String</a>
  toString()
</pre>
```

Del mismo modo se incluirán enlaces a todas las referencias a clases no especificadas en la línea de comandos. Hay que tener en cuenta que los enlaces se realizan suponiendo que la documentación del API se encuentra en el mismo directorio que la documentación que está siendo generada por **javadoc**.

-nodeprecated

Excluye los párrafos que contengan el tag `@deprecated`.

-noindex

Omite el índice del paquete, que por defecto es generado.

-notree

Omite la jerarquía de clases/interface, que por defecto es generada.

-breakindex

Divide el índice en 26 ficheros, uno por cada letra.

-footer *texto*

Coloca el texto que se indique en cada una de las páginas que se generen, inmediatamente antes del tag `</body>`. Si el texto contiene espacios, debe ir encerrado entre comillas simples o dobles.

-version

Incluye las marcas @version, que por defecto son omitidas.

Variables de Entorno**CLASSPATH**

Esta variable de entorno es usada para indicar al sistema la ruta de las clases definidas por el usuario. Los directorios deben estar separados por punto y coma (;). Por ejemplo:

```
C:\users\afq\classes;C:\jdk\classes
```